

Company Name

Target Architecture - Implementation Guide

Ver 0.3 [Draft]

Author

[Ahmed Metwally](#)

16th June 2024

1. Guide Items

1.1 Infrastructure Provision

We going to write **AWS CLI Bash** scripts for below resources

1.1.1 SSH key

Create an ssh key to use it with all EC2 instances.
Store the key on AWS Secrete Manager as a secret.

1.1.2 Security Group

Security group with following ingress rules

- tcp, port 22, open for all
- tcp, all ports, open for security group itself

1.1.3 VPC

Create VPC with following components:

- Two availability zones
- 2 public subnet for each zone
- 2 private subnet in one zone (for data)
- Internet gateway
- Route tables, one for the public subnets, one for the private subnets.

1.1.4 IAM Users, Roles & Policies

All Users, Roles and Policies required for running the infrastructure and CD pipelines.

1.1.4.1 EC2 role:

This role will be attached with all EC2 instances running on the auto scaling group
name = "ec2_service_role"

policies:

- AmazonS3ReadOnlyAccess
- AWSCodeDeployRole

1.1.4.2 CodeDeploy role

This role will be attached to the CodeDeploy

name = "codedeploy_service_role"

policies:

- AmazonS3ReadOnlyAccess
- AWSCodeDeployRole

1.1.4.3 CodePipeline role

This role will be auto generated from the console while creating the pipeline.

1.1.5 Autoscaling Group

Auto scaling group will have the following properties:

- Name pattern: *ENV-serviceName-asg*, (e.g. qc-srv02-asg)
- EC2 tag [name=servicename, value={serviceName}]
- Desired capacity = 2
- Min size = 2
- Max size = 7
- Health check = no
- Policies for increases and decreases instances
- EC2 Launch Template
 - Instance type: t3.micro
 - Main Storage EBS gp3 8GB
 - Key (section 1.1.1)
 - Security groups (section 1.1.2)
 - User Data:

In this script we will do the following:

- Install Dot Net runtime
- Install CodeDeploy agent

1.1.6 Load Balancer

For the auto scaling group

- Type: Network load balancer (NLB)

- Target group port = traffic-port

1.1.7 DNS record

Create DNS A record for the load balancer.

if the environment is not production then record name will be *ENV*srv2.devops90.com

in the production only the record will be srv2.decops90.com without suffix

1.2 CD Pipeline

We will create **systemd** service for the service in each machine.

This should be done in the CodeDeploy AfterInstall script.

1.2.1 CodeDeploy application

Name = *ENV*-srv02-deploy

Compute platform = EC2

1.2.2 Deployment group

Name = *ENV*-srv02-deploy-group

Service role = role from section 1.1.4.2

Deployment type = in-place

Environment configuration

- Auto Scaling groups (the group from section 1.1.5)
- termination hook = yes

Deployment configuration = OneAtATime

Load balancer = disabled

1.2.3 CodeBuild Project

Name = *ENV*-srv02-build

Source = CodeCommit

Environment

- image = Managed image
- OS = Amazon Linux
- Runtimes = Standard

- service role = New service role
- Buildspec = Use a buildspec file

Artifacts

- Type = Amazon S3
- Artifacts packaging = Zip

1.2.4 CodePipeline

Stages:

- Source
 - CodeCommit
 - Branch [master or test] (depends on environment)
 - Trigger when push
- Build
 - CodeBuild project from section 1.2.3
- Deploy
 - CodeDeploy application from section 1.2.1